REA Applied Playbook: @P7RIM3 / Research Desk

Version 2026-10-13 ET · REA `rea-agents` 6.4.0 (https://github.com/morluto/rea) · MCP namespace `user-rea` Sources: S1–S15 in `cheatsheets/SOURCES.md`. Community posts are untrusted claims. Official REA docs win every conflict.

---

0. Ground rules (read first)

From REA `DISCLAIMER.md`: - **Allowed:** security research, malware and incident analysis, vulnerability research and coordinated disclosure, CTFs and education, interoperability, preservation, and debugging or auditing software you **own or are authorized** to examine. - **Not allowed:** bypassing DRM, license checks or paywalls; cheats; bypassing auth or bot protection; **copying proprietary code or assets**; evading AI-provider policies; reselling accounts. - "Public availability … does not by itself establish permission." Authorization to inspect a file does **not** authorize executing it, intercepting its traffic, or touching other parties' systems. - REA output can be wrong. Verify Evidence before acting on it. Analysis output grants **no rights** to the original program.

Corrections to community claims: | Claim | Source | Verdict | |---|---|---| | "Unlocked TradingView premium / free premium indicators" | S8 Bober | ❌ License/paywall bypass. Not allowed. Use only the lawful clean-room pattern in §7. | | $37 → $8k order-book trading | S6 Tenzo | ❌ Anecdote, not an expected return. | | "$200/mo 24/7 AI trading team", $10M NAV dashboard | S10 Khairallah | ⚠️ Demo paper book. Nothing shows live returns. | | "$30/mo X Subscription bundles Grok + Cursor + Premium"; "approvals expire ~10 min" | S11 le0xet | ⚠️ Unverified. Check official pricing and product docs. | | "5 workers for $3/M tokens" | S12 slash1sol | ⚠️ Unverified. Article body not retrieved, and the rate depends on model and input/output mix. | | "62 agents / 29 routines" | S13 Av1dlive | ⚠️ The image says 36 routines and 18 live. About half are paused. | | "Opus 5.5: $4 in / $20 out / $5 cache write / $0.20 cache read; $5.60 vs $43.20" | S14 zodchiii | ✅ Confirmed at https://www.anthropic.com/pricing (Oct 13 ET); the math re-computes exactly (§10.5). | | "REA has 130K GitHub stars" | S14 zodchiii | 🔧 Corrected: 204,103 stars via https://api.github.com/repos/morluto/rea (Oct 13, 5:43 PM ET). | | "143k stars in 2 days", "19.4k stars" | S8, S9 | ⚠️ Social proof. Check the repo yourself. |

---

1. The investigation loop (S1 Codila + official REA)

```
ASK → INSPECT & TRACE → READ EVIDENCE → USE → (unknowns?) → loop
```

1. **Ask** a specific, testable question ("How does the export button build its request?").
2. **Inspect & trace** with `user-rea` tools (local analysis; REA uploads nothing). Optional providers: Ghidra, IDA, Hopper, JADX/JEB, Chrome CDP.
3. **Read Evidence.** Keep three things separate: *observation*, *inference*, and *unknown*. Every claim cites an Evidence record.
4. **Use** it: explain, implement your own code, or write tests. Never paste protected code.
5. **Unknowns** go to `record_unknown`, get resolved with `verify_unknown_resolution`, and get audited with `list_unknowns`.

Definition of done (S4 "no proof, no done"): an exported Evidence bundle (`export_evidence_bundle`), the residual-unknowns list, and a `verify_reconstruction` pass for any spec you claim.

2. Setup & readiness (box only; nothing installed on user machines)

```
node --version          # 22+ (24/26 supported)
npx rea-agents setup    # configures MCP client
rea doctor              # readiness: providers, Chrome CDP, ADB, Ghidra
rea doctor --target <path>
```

Notes: Windows Ghidra is P0 read-only (6.4.0). Bring your own legally obtained IDA, Hopper and JEB licenses.

---

3. Target maps → `user-rea` tools

3.1 Native (Mach-O / PE / ELF)

Step	Tool
Open & session	`open_binary` → `binary_session` → `binary_overview` / `inspect_binary_layout`
Find the feature	`search_strings`, `search_procedures`, `list_strings`, `list_procedures`
Trace	`trace_feature` (literal query) → `xrefs` / `find_xrefs_to_name` → `resolve_containing_procedure`
Read code	`analyze_function`, `procedure_pseudo_code`, `procedure_assembly`, `batch_decompile`, `read_function_instructions`
Call paths	`get_call_graph`, `build_call_path`, `trace_call_path`, `procedure_callers/callees`
Annotate	`set_comment`, `set_address_name`, `set_bookmark`
Apple extras	`inspect_macho`, `get_objc_classes`, `analyze_swift_types`, `inspect_plist`, `inspect_signature`, `project_apple_application_graph`
Windows extras	`inspect_pe_resources`
Runtime (only with authorization to execute)	`prepare_bounded_process_capture` prompt → `capture_process_scenario`, `observe_native_calls`, `correlate_static_and_runtime`
Close	`export_evidence_bundle` → `close_binary`

3.2 JavaScript / Electron

`analyze_javascript_application` (ASAR or dir, no execution) → `inspect_analysis_view` → `trace_application_feature` / `trace_javascript_semantics` → `recover_javascript_sources` → runtime: `list_electron_targets`, `inspect_electron_page`, `capture_electron_scenario`, `observe_javascript_runtime`, `reconcile_javascript_runtime`.

3.3 Web (Chrome CDP, loopback)

`list_browser_targets` → `inspect_web_page` (passive) → `analyze_web_bundle` (chunks, routes, endpoints) → `inspect_web_event_listeners` → `trace_web_module_imports`, `trace_web_source_location` → `observe_web_session` / `observe_web_execution` → `capture_browser_scenario` (only if passive observation can't exercise the app) → `inspect_web_network_capture` (offline HAR/mitm) → `export_web_scripts`, `capture_web_screenshot`, `compare_web_captures`. ⚠️ Stay on public behavior using your own logged-in entitlement. No paywall or bot-protection bypass.

3.4 Android

Static: `inspect_android_package`, `search_android_classes`, `inspect_android_class`, `inspect_android_method`, `trace_android_references`, `decode_android_resources`, `project_android_application_graph`; JEB: `open_jeb_project`, `decompile_jeb_item`. Device (your own device): `list_adb_devices`, `inspect_adb_device`, `list_adb_packages`, `pull_adb_package`, `read_adb_logcat`, `capture_adb_screen`. **Changes the device, so draft and ask first:** `install_adb_package`, `uninstall_adb_package`, `push_adb_file`, `start_adb_app`.

3.5 Firmware

`extract_firmware` → `inspect_firmware_regions` → `open_binary` on extracted images → native flow.

3.6 Smart contracts (EVM)

`inspect_evm_interface` on a **local flat bytecode file**: selectors, args, mutability. No chain/RPC access and no execution. This is audit support, **not** a DeFi trading bot.

3.7 .NET / managed

`inspect_managed_artifact`, `inspect_managed_members`, `compare_managed_members`, `inspect_managed_native_boundaries`, `project_managed_application_graph`.

3.8 Compare & verify (all targets)

`compare_application_versions` prompt → `compare_artifacts`, `compare_functions`, `compare_bundles`, `find_changed_behavior` → `build_reconstruction_obligation_ledger` → `evaluate_reconstruction_coverage` → `verify_reconstruction`.

---

4. Copy-paste prompt pack

The six official MCP prompts are `investigate_feature`, `compare_application_versions`, `verify_reconstruction`, `trace_crash`, `audit_residual_unknowns`, and `prepare_bounded_process_capture`.

```
[INVESTIGATE] Using user-rea, investigate_feature "<feature>" in <path>. I own/am authorized
to analyze it. Static only. Separate observations / inferences / unknowns, cite Evidence IDs,
record_unknown for gaps, export_evidence_bundle to ./evidence/<name>.json.

[DIFF] compare_application_versions <old> <new>. List behavior changes with Evidence;
flag anything security-relevant. No execution.

[WEB] list_browser_targets on loopback CDP, inspect_web_page + analyze_web_bundle for <origin>
(my own logged-in session). Map routes/endpoints for feature <X>. Passive only; no paywall bypass.

[CRASH] trace_crash with symptom "<log line>" against <binary>. Propose a bounded capture plan
(prepare_bounded_process_capture) but do not run it until I approve.

[AUDIT] audit_residual_unknowns: list open unk_ heads by severity; which can be resolved
statically?
```

```
[SPEC→BUILD] From exported Evidence write an independent behavioral spec (no copied code),
then verify_reconstruction against it. Report coverage and failing claims.
```

---

5. Unknowns, Evidence & memory routing (S5)

Item	Route	Owner / expiry
Evidence bundle `ev_…`	Project folder (`evidence/`); approved team memory once reviewed	Investigator; re-check on target version change
Residual unknowns `unk_…`	Live in REA session plus private note	Investigator; until resolved
Decisions (spec, legal scope, authorization)	**Approved team memory** with source, scope, owner and review date	You; review quarterly
Live market data, prices, balances	**Live source only**, never memorized	—
Secrets, API keys, captured credentials	**Never** in memory or notes; redact HAR captures	—

Gate: is it stable, shared, and sourced? If yes, it goes to team memory. If not, it stays a private note or is fetched live.

---

6. Team manager pattern (S4 Akarixbt, S10 Khairallah, S11 le0xet)

Brief every worker with: **goal → brief/scope → authorized targets → verification ("no proof, no done") → approval boundaries.**

Overnight stoplines (S11): - **SCOUT** detects change (new REA release, new target version, new X mentions) and decides whether to wake anyone. - **EXEC** does reversible work alone: static REA analysis, diffs, Evidence export, drafts. - **RISK** gets anything irreversible (posts, emails, orders, ADB installs, runtime capture on third-party systems) and turns it into a **draft or queued approval only**.

A chief-of-staff router (S10) should do routing and summaries, not trading.

---

7. Authorized trading-software & quant-desk workflow (S6, S7, S8)

Pipeline (S7): **authorized target → REA + MCP Evidence → independent model → quant validation → risk gates → paper trade → controlled review.**

Net-return model: **R_net = R_gross − C_fees − C_slippage − C_market-impact − C_ops.** Never report a gross return alone.

Lawful version of S8 "rebuild a charting workstation": 1. Observe the **public** UI behavior of tools you're entitled to use: `inspect_web_page`, `capture_web_screenshot`. 2. Write a **behavioral spec** in your own words (chart types, indicator math from public literature). 3. Build **your own** charting and backtest engine with fees, slippage, and drawdown. 4. Test it with `verify_reconstruction` against your own spec. Do **not** port proprietary indicator code, scripts, or premium-only features.

Order-book discipline (S6, applied): check depth before size, don't trade a thin book, keep a cash buffer, and use limit orders.

Your book: Kraken **spot** only, about $108 in BTC/ETH/SOL/ADA, under the Research Desk limits. Paper trade first. No leverage. **No unattended live orders.** No guaranteed returns. REA output is research, not a trading signal.

---

8. Cost-efficient multi-worker REA runs (S12, S11, S13)

S12's "$3/M tokens" claim is **unverified**: the article body wasn't retrievable. Verify against current provider pricing (input vs output, cached vs uncached) before you plan around it.

Lean rules that hold regardless of price: 1. **Do the heavy work locally, pay little for tokens.** REA analysis runs locally. Send the model compact Evidence (IDs plus snippets), not whole decompiles. Prefer `trace_feature`/`search_*` over `batch_decompile` of everything. 2. **Tier workers by job.** Use a cheap or fast model for scouts and triage (string search, diff summaries) and a strong model only for spec writing and verification. 3. **Parallelize by target, not by step.** One worker per artifact or version, each with its own `binary_session`, merged by a chief of staff. 4. **Cap each worker:** max tool calls, max decompiled functions, and a token budget per run. Stop at the residual-unknowns threshold and escalate. 5. **Cut wasted wakeups (S11).** Scouts exit early when nothing changed. Every wakeup costs budget. 6. **Prune routines (S13).** About half of the 36 routines were paused. Disable routines that haven't produced a decision in 2 weeks. 7. **Measure:** log tokens, tool calls, and "findings per $" per routine. Re-check flat-rate vs metered pricing monthly, because it depends on volume.

---

9. Lean agent + routine layout for @P7RIM3 (S13 adapted)

You already run **Research Desk** and **Signal**. Add three narrow workers rather than 62.

Agent	Role	Routines	Mode
Research Desk (existing, chief of staff)	Routes, merges Evidence, owns team memory, morning brief	Morning brief 08:00 wkdays; weekly review Mon 09:00	Live
REA Scout (new)	Watches morluto/rea releases and new versions of authorized targets; runs static diff	Daily 07:30 check; runs `compare_application_versions` only on change	Live, exits early
REA Analyst (new, on demand)	Runs §1 loop / §4 prompts, exports Evidence, lists unknowns	Triggered by Scout or you	On demand
Signal (existing)	Market and news context for the Kraken spot book; paper-trade journal	am + pm digest	Live; drafts only, never orders
X Content Lead (new)	Turns *verified* REA findings into X post/thread **drafts** (what changed, how verified, Evidence-backed; no proprietary code, no "free premium" claims)	Weekday 08:55 draft from previous day's Evidence; weekly "REA field notes" Fri	**Drafts only. You approve every post.**

REA → X wiring: Scout detects → Analyst produces Evidence + unknowns → Research Desk reviews (stable/sourced? authorized target?) → Content Lead drafts a post citing the method, not protected internals → **you approve** → post. Live market numbers come from Signal at draft time, never from memory.

Running live vs paused lesson (S13): only 18 of 36 routines were live. Start with the 5 above, add a routine only when a manual task repeats 3+ times, and pause any routine with no approved output in 2 weeks.

---

10. Read-only RE workspace + findings log (S14 zodchiii, S15 RoundtableSpace)

S14 is a Claude Code guide. Its method maps onto `user-rea` directly. Use it only for targets you **own or are authorized** to analyze (§0). @zodchiii's follow-up post pitching it to "reverse-engineer any paid closed-source app" is **excluded**. S15 is a later, uncredited, truncated summary of S14's steps 1–2 and adds nothing new.

10.1 Loop → REA mapping

S14 step	REA on the box
Read-only target (repo, HAR, bundle, binary)	`open_binary` / `open_jeb_project` on `target/`; REA analysis is static and doesn't run the target. Check with `binary_session`.
Mechanical tools first (webcrack, Ghidra, REA)	`binary_overview`, `inspect_binary_layout`, `list_procedures`, `list_strings`, `analyze_javascript_application`, `recover_javascript_sources` (Wakaru, REA's equivalent of webcrack)
Model analysis (trace, rename, explain)	`trace_feature`, `xrefs`, `get_call_graph`, `trace_javascript_semantics`, `analyze_function`, `procedure_pseudo_code`, `batch_decompile`
Findings log CONFIRMED / GUESS with file:line	CONFIRMED = cites an Evidence ID plus address or file:line. GUESS = `record_unknown`. Read back with `list_unknowns`.
Verified? No → new evidence with a cached follow-up	Keep **one session**. Target the gap with `inspect_address_context`, `read_bytes`, `trace_native_values`, then `verify_unknown_resolution` / `update_unknown`.
Yes → ARCHITECTURE.md	Write it from `get_evidence_bundle`, then `export_evidence_bundle` to `notes/evidence.json`. Open questions come from `list_unknowns`.
Spec/client (openapi.yaml, client.ts, .ksy, parse.py)	Write an independent spec, then `build_reconstruction_obligation_ledger` → `evaluate_reconstruction_coverage` → `verify_reconstruction`

10.2 Workspace on the box

```
/workspace/re/<target>/
  target/   # chmod -R a-w; analysis input only
  notes/    # FINDINGS.md, ARCHITECTURE.md, evidence.json, spec/
  CLAUDE.md # rules: tag every claim CONFIRMED(ev_/file:line) or GUESS; never edit target/
```

The Claude Code `.claude/settings.json` in S14 (allow `Read(./target/**)` and `Edit(./notes/**)`; deny `Edit(./target/**)`, WebFetch, WebSearch, curl and wget) is a convenience, not a security boundary. On the box, make `target/` read-only at the filesystem level and run any execution only through REA's bounded capture tools after approval (§3.1).

10.3 Workflows

- **A. Codebase map:** find the entry points (`list_procedures`/`search_procedures`, or `analyze_javascript_application` for JS) → trace the 5 main data flows (`trace_feature`, `trace_call_path`, `trace_application_feature`) → diagram → flag hardcoded secrets (`search_strings` regex) and dead code (procedures with no `procedure_callers`) → ARCHITECTURE.md.
- **B. API rebuild:** use **your own** HAR, with Authorization headers, cookies and tokens redacted *before* analysis (failure mode 2) → `inspect_web_network_capture` (offline, contacts nothing) + `analyze_web_bundle` for endpoints → OpenAPI 3.1 + typed TS client. Only for APIs you're entitled to use.

- **C. Deobfuscation:** `recover_javascript_sources` (or webcrack) → rename, logging each rename in `notes/RENAMES.md` → for native targets, `set_addresses_names` + `annotate_native_function` (readback verified) → spot-check about 10% against `procedure_pseudo_code`.
- **D. Binary analysis:** Ghidra/IDA/Hopper through `open_binary`, Android through JADX (`inspect_android_*`) or JEB (`decompile_jeb_item`). **Checkpoint after 30 functions** and confirm the direction with you before continuing.
- **E. File formats:** use 3–5 of your own samples → `open_binary` + `read_bytes` / `inspect_firmware_regions` → `format.ksy` (Kaitai) + `parse.py` → check that every sample parses.

10.4 Prompt

```
[S14-RE] Target ./target/<x> (I own/am authorized). Read-only; write only to ./notes.
Run mechanical user-rea tools first, then explain. Log each claim in notes/FINDINGS.md as
CONFIRMED (Evidence ID + file:line/address) or GUESS (+record_unknown). Stop after 30
functions and report direction. Final: ARCHITECTURE.md + export_evidence_bundle.
```

10.5 Cost check: ✅ confirmed (Oct 13 2026 ET)

Opus 5.5 at https://www.anthropic.com/pricing costs $4 input, $20 output, $5 cache write and $0.20 cache read per MTok (5-minute TTL). Recomputed for a 400K-token context with 25 follow-ups and 80K output tokens in total: - **Cached:** $2.00 write + $2.00 reads + $1.60 output = **$5.60** - **Fresh:** 26 × $1.60 = $41.60 + $1.60 output = **$43.20**

Both match. The assumptions are follow-ups within 5 minutes of each other (otherwise a $2 re-write each time) and question tokens ignored. Same lesson as §8: keep one warm session per target.

10.6 Failure modes (S14)

1. Skipping the CONFIRMED/GUESS tags.
2. Analyzing raw HARs that still contain tokens.
3. Letting the model write to `target/`.
4. Opening a fresh session per question.
5. Trusting renames without spot-checking about 10%.

REA claim check: S14 says 130K stars, and the GitHub API shows **204,103** (https://github.com/morluto/rea, Oct 13 2026, 5:43 PM ET). 🔧 Corrected.

---

11. Gaps

- S1 Codila: page 1 only. S13: page 2/2 only. Page 1 is not in the payload; the user's attachment is the same page 2.
- S6, S8–S11 are videos; previews and frames saved and text taken from the notes. The S10 quoted article returned 404; the S12 article body is not retrievable.
- S9 (local ElevenLabs alternative) is off-topic; it's kept only as a star-count-hype example.
- All pricing claims are unverified (§0).
- S15 is cut off at step 3, and its photo wasn't opened, so whether it copies S14 is inferred from timing and wording only.